Systems Track

Interview Preparation Series • Professional Edition

Distributed Systems

07

Compiled with LaTeX • June 17, 2026

harsha.somisetty01@gmail.com

In This Edition

> **Audience:** Senior engineers preparing for FAANG system-design and depth loops. **Goal:** First-principles mastery of every distributed systems concept examiners probe. **How to use:** Read deeply once, then use the Revision Cheat Sheet to refresh before each interview.

1 Overview --- What it is

A **distributed system** is a collection of autonomous computing nodes that communicate over a network and appear to users as a single coherent system.

Formal definition (Lamport): "A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable."

Key properties:

- Nodes run concurrently and independently
- Nodes communicate only via message passing (no shared memory)
- Nodes can fail independently — partial failure is the norm
- There is no global clock; clocks drift

Why it's hard in one sentence: Nodes fail, networks partition, and clocks lie — all simultaneously, silently, and at the worst possible moment.

2 Why It Exists

Distributed systems are necessary when a single machine's limits are reached:

Limit	Distributed Solution
CPU/memory wall	Horizontal scale-out across many machines
Storage capacity	Distributed file systems (HDFS, GFS)
Geographic latency	Data replicated to edge / regional DCs
Fault tolerance	Replicate so no single failure kills the system
Throughput ceiling	Partition (shard) data across nodes

Single machines are simpler but bounded. Netflix can't serve 250M users from one server. Google can't index the web on one disk. Amazon can't guarantee 99.99% uptime on one machine.

The moment you cross machine boundaries, you inherit: network unreliability, partial failures, clock skew, and the need to coordinate — the core distributed systems problems.

3 Why FAANG Cares (be specific per company)

3.1 Google

- **Spanner** (globally distributed RDBMS, TrueTime, external consistency) — invented because Bigtable and Megastore were hard to use correctly
- **Chubby** (lock service, Paxos-based) — foundation of all Google coordination
- **MapReduce, Colossus, Borg** — entire infrastructure is distributed
- Google interviews heavily test **consensus, consistency models, and CAP trade-offs**. Expect: "Design Spanner's TrueTime" or "How does Chubby work?"

3.2 Meta (Facebook)

- **Cassandra** (originally built at FB for Inbox search) — AP system with tunable consistency
- **TAO** (distributed graph cache for social graph) — read-heavy, eventual consistency
- **ZippyDB**, **Memcached/mcrouter** — caching layer
- Meta cares about **eventual consistency, caching patterns, and large-scale fan-out**

3.3 Amazon / AWS

- **DynamoDB** (Dynamo paper, AP, consistent hashing, vector clocks, sloppy quorum)
- **Aurora** (CP, quorum writes, storage disaggregation)
- **Kinesis** (Kafka-like streaming)
- Amazon interviews emphasize **Dynamo-style trade-offs, eventual consistency, and operational realities**

3.4 Microsoft

- **Azure Cosmos DB** (multi-model, five consistency levels, PACELC poster child)
- **Azure Service Bus**, **Event Hubs** (Kafka-like)
- Microsoft asks about **consistency models** (they literally offer five), PACELC, and geo-replication

3.5 Apple / Netflix / Uber / LinkedIn

- All run Kafka for event streaming
- All use service meshes (Envoy, Istio) with distributed service discovery
- All depend on consistent hashing for caching and load balancing
- Interviews test **Kafka deep dives, service discovery, distributed caching**

4 The 8 Fallacies of Distributed Computing

Originally articulated by Peter Deutsch (Sun Microsystems, 1994). Every fallacy represents an assumption that kills distributed systems.

#	Fallacy	Why It Bites You
1	**The network is reliable**	Packets get dropped, routers fail, links saturate. TCP retries help but add latency; still doesn't guarantee delivery at the application layer.
2	**Latency is zero**	Even same-DC calls: ~0.5ms. Cross-continent: 150ms+. Synchronous RPC chains multiply latency. One N-hop call = sum of all latencies.
3	**Bandwidth is infinite**	Large payloads, bulk transfers, or fan-out can saturate NICs or egress. Protobuf over JSON, streaming over batching often necessary.
4	**The network is secure**	Man-in-the-middle, rogue nodes, spoofing. Requires TLS, mTLS, service identity. Especially critical in multi-tenant clouds.
5	**Topology doesn't change**	Nodes are added, removed, rebalanced. IP addresses change. Service discovery must be dynamic, not static config.
6	**There is one administrator**	Multi-team, multi-cloud, multi-region. Policies conflict. Schema migrations, firewall rules, and ops procedures must be coordinated across orgs.
7	**Transport cost is zero**	Serialization, encryption, compression, and network egress all cost CPU and money. Cross-AZ bandwidth is billed.
8	**The network is homogeneous**	Different NIC speeds, MTUs, protocols, OS network stacks. Kubernetes pods, bare metal, VMs, edge nodes — all in one system.

Interview takeaway: Bring up the 8 fallacies when asked "What makes distributed systems hard?" — it signals systems maturity. The most interview-relevant are **#1 (reliability) + #2 (latency) + topology change (#5)**.

5 Core Concepts

5.1 CAP Theorem

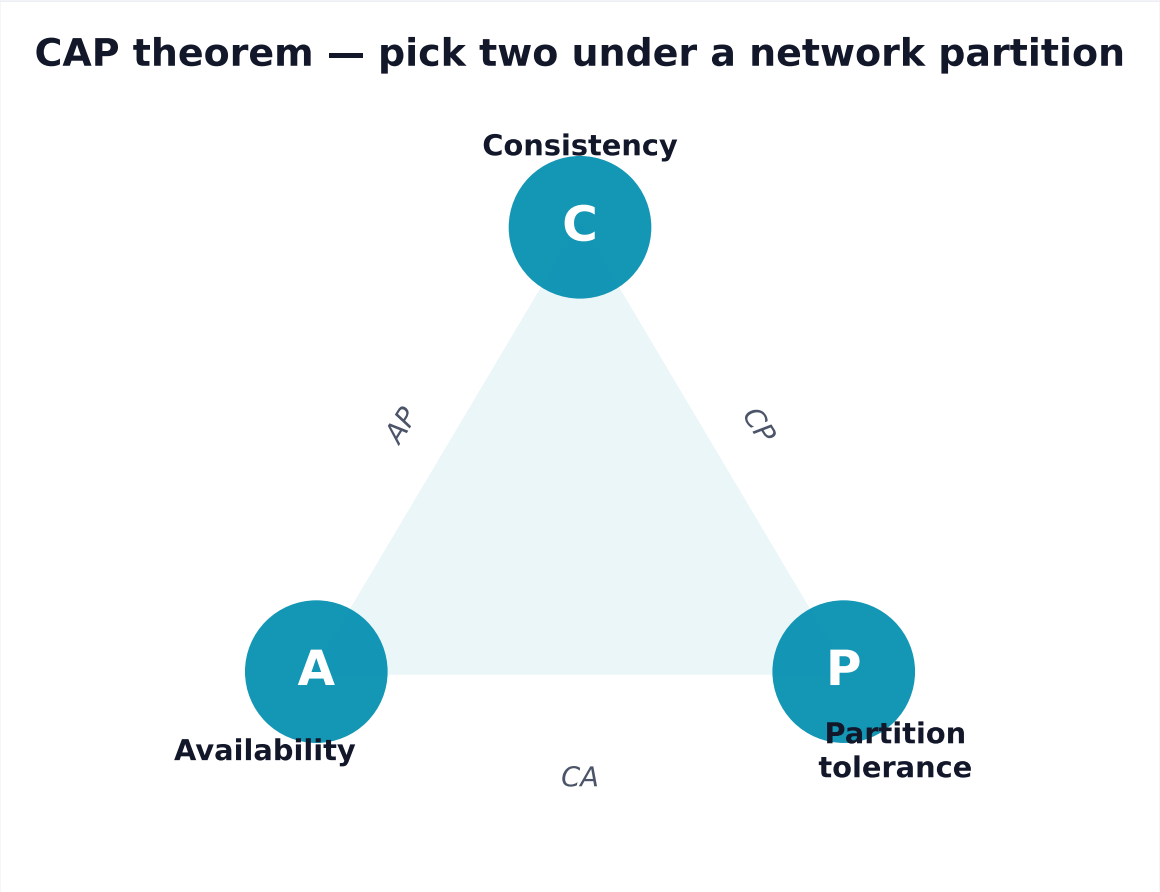


Figure 1. When the network partitions, a distributed store must sacrifice either consistency (AP) or availability (CP). Partition tolerance is non-negotiable.

Statement (Brewer, 2000; formalized by Gilbert & Lynch, 2002):

> In the presence of a network **P**artition, a distributed system must choose between **C**onsistency and **A**vailability.

The three properties:

Property	Meaning
Consistency (C)	Every read receives the most recent write or an error (linearizability)
Availability (A)	Every non-failing node returns a response (possibly stale)
Partition Tolerance (P)	The system continues operating despite arbitrary message loss between nodes

The key insight: Partition tolerance is NOT optional in real systems. Networks partition.

You **must** tolerate partitions. Therefore the real choice is:

- **CP systems:** Sacrifice availability during partitions. Return errors rather than stale data. Examples: ZooKeeper, etcd, HBase, Spanner, MongoDB (primary reads)
- **AP systems:** Sacrifice consistency during partitions. Return possibly-stale data rather than errors. Examples: Cassandra, DynamoDB (default), CouchDB, DNS

CA systems: Only possible in single-node (no partitions possible by definition). RDBMS on a single machine is "CA" — but this is trivial in distributed context.

Common misconception: CAP says nothing about latency, throughput, or behavior under normal (non-partition) operation. PACELC extends this.

Interview takeaway: "CA is not a real choice in distributed systems — partitions happen, so you always pick CP or AP. The choice is: do you return an error or stale data when the network splits?"

5.2 PACELC

Extension by Daniel Abadi (2010): Captures what happens when there's NO partition too.

> **If Partition:** choose between **A**vailability and **C**onsistency **ELse (normal operation):** choose between **L**atency and **C**onsistency

```python
P → A vs C
ELC → L vs C
```

System	Partition choice	Normal-ops choice	Category
DynamoDB	A	L	PA/EL
Cassandra	A	L	PA/EL
Spanner	C	C	PC/EC
MongoDB	C	C	PC/EC
ZooKeeper	C	C	PC/EC
Cosmos DB	Tunable	Tunable	All categories depending on config

Why PACELC matters: CAP only says what to do during a partition (rare). PACELC captures the latency vs. consistency trade-off that happens on EVERY SINGLE READ/WRITE. In practice, the ELC trade-off is more important day-to-day.

Interview takeaway: "CAP is about rare partition events. PACELC is about the trade-off you make on every operation. Most high-traffic systems choose AP/EL — they prefer low latency and availability over strong consistency."

5.3 Consistency Models

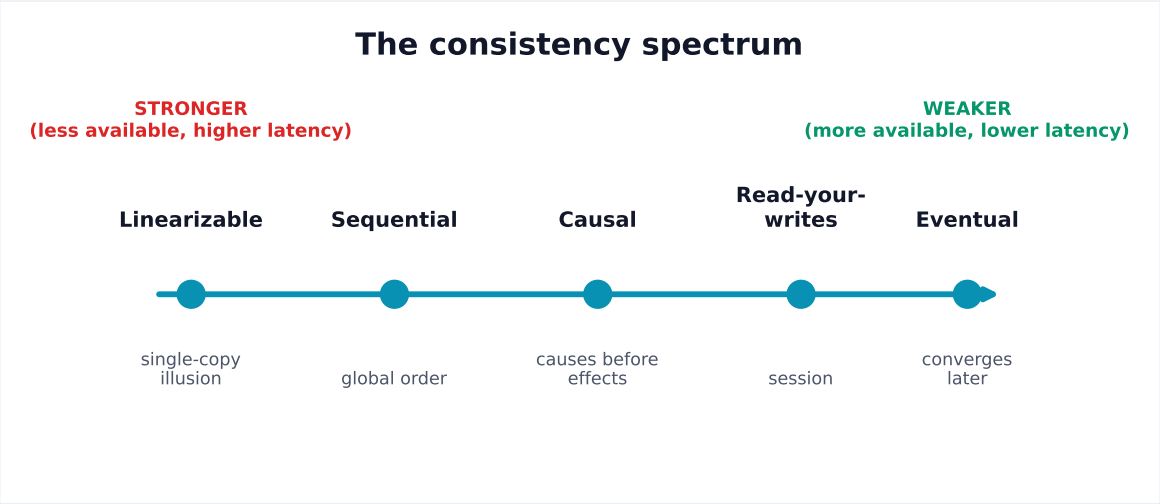


Figure 2. Consistency is a dial, not a switch. Stronger models are easier to reason about but cost availability and latency under partitions — pick per use-case.

From strongest to weakest:

Model	Guarantee	Cost	Examples
Linearizability (Strong)	All ops appear instantaneous at some point between call and return. Total order matches real time.	High latency (synchronous coordination)	Spanner, etcd, Zookeeper
Sequential Consistency	All ops appear in some sequential order consistent with each process's order. No real-time requirement.	High	Research systems
Causal Consistency	Causally related ops seen in order by all nodes. Concurrent ops may be seen in different orders.	Medium	COPS, MongoDB causal sessions
Read-your-Writes	After you write X, you always read X (from any node). Others may see stale data.	Low-medium	Session consistency, sticky sessions
Monotonic Reads	If you read value V, you'll never read a value older than V.	Low	Cassandra with session-level consistency
Monotonic Writes	Writes from one process are applied in order at all nodes.	Low	Kafka per-partition ordering
Eventual Consistency	Given no new writes, all replicas converge to the same value. No timing guarantee.	Lowest	DynamoDB default, DNS, Cassandra default

Interview mental model:

```
Strongest                                    Weakest
Linearizable → Sequential → Causal → Read-your-writes → Eventual
```

Key distinctions examiners probe:

- **Linearizability vs Serializability:** Linearizability is about single operations (real-time ordering). Serializability is about transactions (equivalent to some serial execution). Strict Serializability = both.
- **Eventual vs Strong:** Eventual is fine for social media likes. Strong is required for bank balances and inventory counts.

5.4 Consistent Hashing

Problem: Naive hash-based sharding (node = hash(key) % N) requires remapping ALL keys when N changes. Catastrophic for large caches or DBs.

Solution: Place both nodes and keys on a virtual ring [0, 2^32). A key maps to the nearest node clockwise.

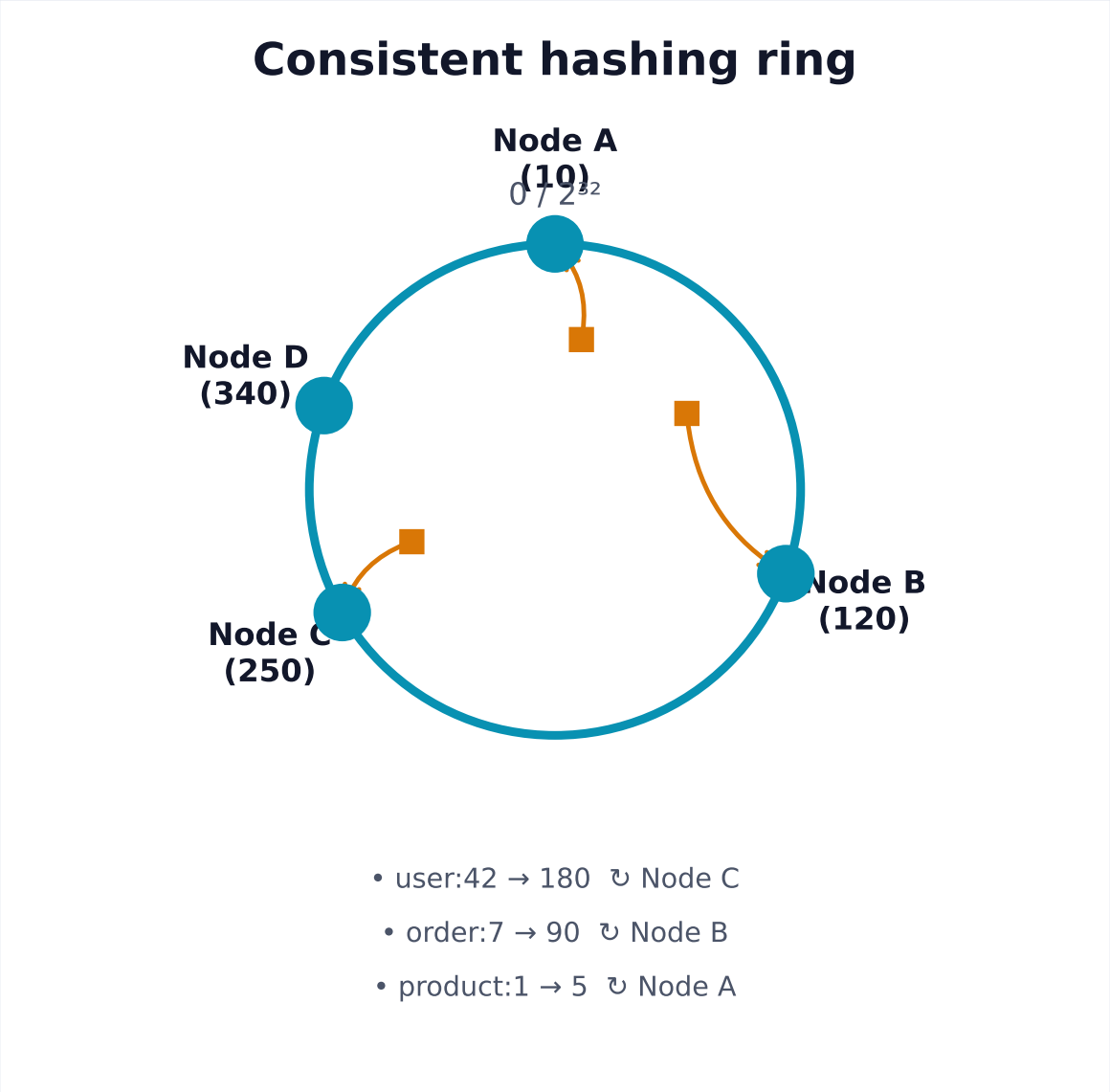


Figure 3. Keys map to the first node clockwise on a hash ring. Adding/removing a node remaps only 1/N of keys (not all of them as in naive modulo hashing).

Virtual nodes (vnodes): Each physical node owns multiple positions on the ring. This improves load distribution and makes rebalancing smoother.

```
Physical: NodeA, NodeB, NodeC (3 nodes)
Virtual:  A1, A2, B1, B2, C1, C2 scattered around ring

Adding NodeD: It claims some positions from A, B, and C equally
```

Why vnodes? Without them, if NodeA sits between 0-33% of the ring and NodeB sits between 33-100%, NodeB serves 2x more traffic. Vnodes balance this.

Interview takeaway: "Consistent hashing limits key remapping to O(K/N) when nodes join/leave vs O(K) for naive modular hashing. Virtual nodes add uniform load distribution."

Real usage: Amazon Dynamo, Apache Cassandra, Memcached (ketama), CDN edge node selection.

5.5 Quorum (R + W > N)

Setup: N total replicas. Write to W replicas. Read from R replicas.

Quorum rule: R + W > N guarantees at least one node in the read set has the latest write.

```
Quorum intersection example (N=3):

Write quorum W=2: Write goes to nodes {A, B}
Read quorum R=2:  Read comes from nodes {B, C}

Overlap = {B} — B always has the latest value.
Proof: W + R = 4 > N = 3, so overlap of at least 1.

     [A] ← Write
     [B] ← Write ← Read
     [C]         ← Read
```

Common configurations:

R	W	N	Characteristic
1	N	N	Write-heavy: slow writes, fast reads
N	1	N	Read-heavy: fast writes, slow reads
(N+1)/2	(N+1)/2	N	Balanced (majority quorum)
1	1	N	No quorum! Fastest but inconsistent

Sloppy quorum (Dynamo): During partition, accept writes to any available node (not necessarily the "correct" N nodes). Use **hinted handoff** to replay writes to original nodes when they recover. Boosts availability but risks temporary inconsistency.

Interview takeaway: "R+W>N guarantees read-write overlap, making strong consistency possible without a single coordinator. Sloppy quorum breaks this guarantee for higher availability — AP trade-off."

5.6 Consensus: Raft

Problem: How do distributed nodes agree on a single value (or sequence of values) despite failures?

Raft (Ongaro & Ousterhout, 2014) — designed to be understandable vs Paxos.

Raft: Three Roles

```
Leader:    Handles all client reads/writes. Sends heartbeats.
Follower:  Passive. Responds to leader and candidates.
Candidate: Requests votes during election.

State machine:
   Follower → (election timeout) → Candidate → (majority votes) → Leader
   Candidate → (discovers leader)  → Follower
   Leader    → (higher term seen)  → Follower
```

Raft: Terms

Each "epoch" is a **term** (monotonically increasing integer). Term acts as a logical clock. Stale leaders detecting a higher term immediately step down.

Raft: Leader Election

```
Step 1: Follower's election timeout fires (150-300ms random)
Step 2: Follower increments term, becomes Candidate, votes for self
Step 3: Sends RequestVote RPC to all other nodes
Step 4: Each node grants at most one vote per term (first-come-first-served)
        BUT only if candidate's log is at least as up-to-date as voter's log
Step 5: Candidate receiving majority votes → becomes Leader
Step 6: Leader immediately sends AppendEntries (empty = heartbeat) to assert authority

Split vote: Multiple candidates, no majority. Election timeout fires again with new term.
Random timeouts make split votes rare.
```

Raft: Log Replication

```
Client → Leader (append "SET x=5")
Leader appends to local log (uncommitted)
Leader sends AppendEntries to all Followers in parallel
Followers append to their logs, reply OK
Leader receives majority ACKs → commits entry
Leader applies to state machine, returns to client
Leader notifies followers of commit in next heartbeat
Followers apply committed entries to their state machines

Log structure:
Index: | 1 | 2 | 3 | 4 | 5 |
Term:  | 1 | 1 | 2 | 3 | 3 |
Cmd:   |SET|SET|DEL|SET|SET|
             ↑
             committed up to here

Leader always checks prevLogIndex and prevLogTerm before appending
This ensures followers' logs never diverge
```

Raft: Safety Guarantee

Leader Completeness: A leader always has all committed entries. Guaranteed by the election constraint: a candidate cannot win if its log is less up-to-date than the majority. "Up-to-date" = higher last term, or same last term and longer log.

Log Matching: If two entries in different logs have the same index and term, the logs are identical up to that index.

Raft: Cluster Changes

Joint consensus: When adding/removing nodes, Raft uses a two-phase config change to avoid split-brain (two independent majorities).

5.7 Consensus: Paxos

Paxos (Lamport, 1989/1998) — the original consensus algorithm. Notoriously hard to understand and implement.

Single-Decree Paxos: Two Phases

Phase 1: Prepare/Promise

```
Proposer → sends Prepare(n) to majority of Acceptors   (n = proposal number)
Acceptor → if n > any seen before:
             Promise not to accept any proposal < n
             Return highest-numbered proposal previously accepted (if any)
```

Phase 2: Accept/Accepted

```
Proposer → sends Accept(n, v) to majority
           (v = value from highest-numbered promise, or proposer's own if none)
Acceptor → if no promise to ignore n, accept it
           Sends Accepted to Proposer and Learners
Proposer → value is chosen when majority accept
```

Why Paxos is hard:

1. **Single-decree only** — choosing one value. Multi-Paxos (choosing a log) requires additional hacks
2. **Liveness not guaranteed** — two proposers can keep preempting each other (FLP impossibility)
3. **No distinguished leader** — Multi-Paxos assumes a leader but doesn't specify how to elect one
4. **Gap-filling** — what if an instance has no chosen value? Holes in the log
5. **Implementation complexity** — Google Chubby paper says it took 2 years to get right

Raft vs Paxos Comparison

Aspect	Raft	Paxos (Multi)
Designer	Ongaro & Ousterhout (2014)	Lamport (1989)
Designed for	Understandability	Correctness proof
Leader	Explicit, strong	Implicit assumption
Log holes	Impossible (sequential)	Possible, need gap-fill
Config changes	Joint consensus	Ad-hoc
Implemented in	etcd, CockroachDB, TiKV	Chubby, Zookeeper (ZAB), Spanner
Understandability	High	Low (Lamport: ”choose trivially simple”)
Performance	Comparable	Comparable

Interview takeaway: ”Paxos is proven correct but painful to implement. Raft achieves equivalent safety with explicit leader election and sequential log, making it far easier to implement correctly. For interviews, know Raft deeply.”

5.8 Leader Election

Beyond Raft, general leader election approaches:

Bully Algorithm:

1. Node detects leader failure
2. Sends Election message to all higher-ID nodes
3. If no response, declares itself leader
4. Highest-ID node always wins

› Problem: O(N^2) messages, highest ID isn't always best

Ring Election (Chang-Roberts):

1. Each node passes its ID clockwise around a ring
2. Each node forwards only IDs higher than its own
3. When a node receives its own ID, it's the winner

› O(N) messages in best case, O(N^2) worst

ZooKeeper/etcd approach (most practical):

› Nodes write an ephemeral sequential znode

- The node with the smallest sequence number is the leader
- Others watch the node just before them (not the leader directly) — avoids herd effect

Interview takeaway: "In practice, leader election is solved by Raft/Paxos or by external coordination services like etcd/ZooKeeper. Bully and ring are textbook algorithms rarely used in production."

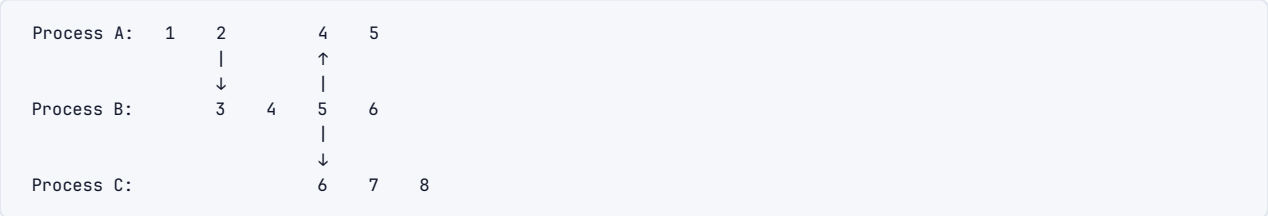
5.9 Vector Clocks and Lamport Timestamps

The problem: Distributed nodes have no shared clock. We need to reason about event ordering without a global clock.

Lamport Timestamps

Rules:

1. Each process starts counter at 0
2. On each local event: increment counter
3. On send: increment, attach counter to message
4. On receive: max(local, message_counter) + 1



Limitation: Lamport timestamps give a total order but cannot distinguish concurrent events. If A.ts < B.ts, A happened-before B OR they are concurrent — you can't tell which.

Vector Clocks

Each node maintains a vector of counters, one per process.

Rules:

1. On local event at node i: increment V[i]
2. On send from node i: increment V[i], attach full vector
3. On receive at node j from i: V[j] = max(V[j], V_msg) element-wise, then increment V[j]

Comparison:

- V_a < V_b (a happened-before b): V_a[i] ≤ V_b[i] for all i, strict for at least one
- Concurrent: neither V_a < V_b nor V_b < V_a

```
Example (3 nodes A, B, C):
A sends to B: A=[1,0,0]
B receives:   B=[1,1,0]
B sends to C: B=[1,2,0]
C receives:   C=[1,2,1]
A acts:       A=[2,0,0]

A=[2,0,0] vs B=[1,1,0]: neither < other → CONCURRENT
C=[1,2,1] vs A=[2,0,0]: C[0]=1 < A[0]=2 but C[1]=2 > A[1]=0 → CONCURRENT
```

Interview takeaway: "Vector clocks detect causality between events in distributed systems. DynamoDB used them (originally) to detect conflicting writes. They scale as O(N) per message. For large N, dotted version vectors or hybrid logical clocks are used."

5.10 Two-Phase Commit (2PC) and Alternatives

2PC — classic distributed transaction protocol.

2PC Flow

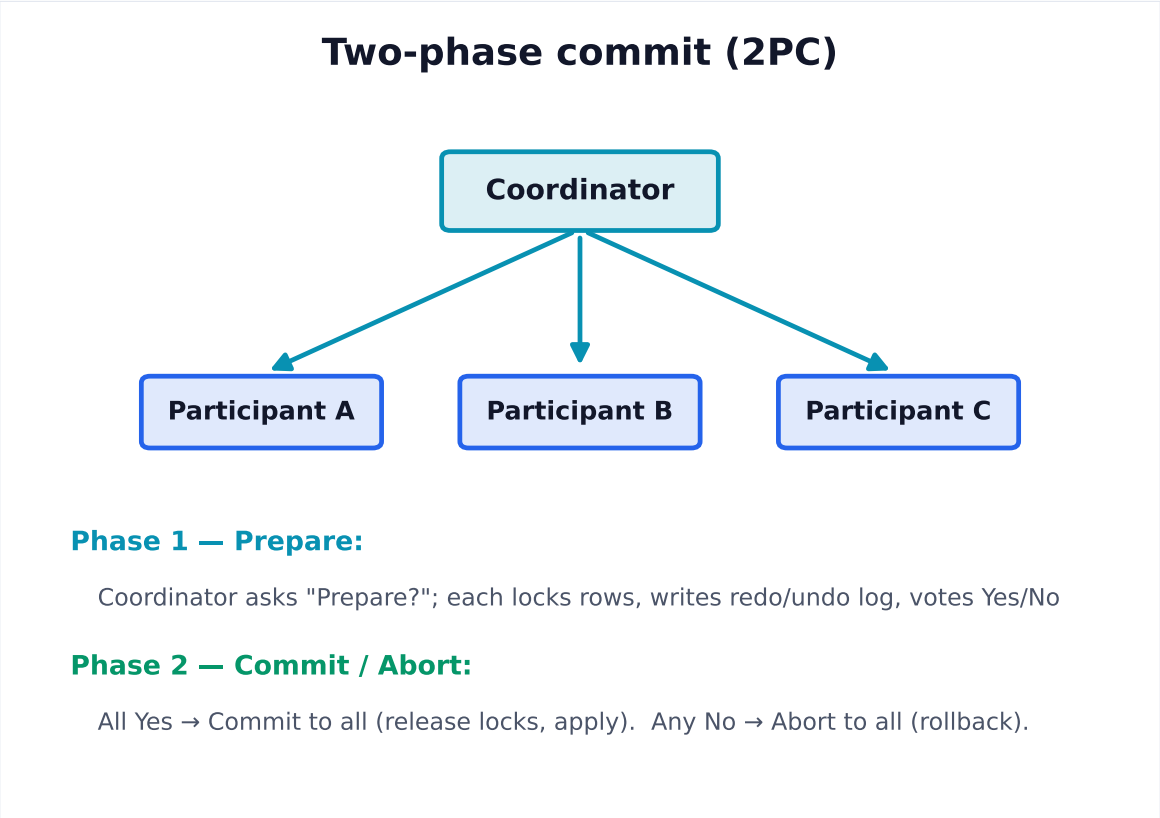


Figure 4. 2PC gives atomic commit across nodes: a prepare vote then a commit/abort decision. Its weakness — if the coordinator dies after prepare, participants block holding locks.

2PC Problems:

Problem	Description
Blocking	If coordinator crashes after Phase 1, participants are stuck holding locks indefinitely
Single point of failure	Coordinator crash = system halted until coordinator recovers
Not partition-tolerant	Cannot make progress during network partition
Performance	2 round trips + synchronous lock holding = high latency

Three-Phase Commit (3PC)

Adds a **Pre-Commit** phase between Prepare and Commit. Allows participants to safely abort if coordinator fails after Pre-Commit. Theoretically non-blocking but rarely used because: complex, still fails under network partition, and adds a 3rd round trip.

Production reality: 2PC is used in databases (XA transactions) but rarely across microservices. Too slow, too fragile.

5.11 Distributed Transactions and Saga Pattern

Saga Pattern — alternative to 2PC for long-running distributed transactions.

Concept: Break a transaction into a sequence of local transactions. Each local transaction publishes an event or message. If any step fails, execute **compensating transactions** to undo previous steps.

```
Order Saga:
  1. Create Order         → if fails: N/A (first step)
  2. Reserve Inventory    → if fails: Cancel Order
  3. Process Payment      → if fails: Release Inventory, Cancel Order
  4. Arrange Shipping     → if fails: Refund Payment, Release Inventory, Cancel Order
  5. Confirm Order        → success

Compensating transactions (run on failure):
  Cancel Order → Release Inventory → Refund Payment → Cancel Shipping
```

Two Saga implementations:

Type	How	Pros	Cons
Choreography	Each service listens to events and acts	Decoupled, no central coordinator	Hard to track flow, debugging complex
Orchestra	Central saga orchestrator tells each service what to do	Easy to track, clear flow	Orchestrator is a bottleneck/SPOF

2PC vs Saga:

Aspect	2PC	Saga
Consistency	ACID (strong)	Eventually consistent
Latency	High (sync, locks)	Low (async)
Failure handling	Automatic rollback	Compensating transactions
Coupling	Tight (all in one TX)	Loose (events/messages)
Use case	Same DB, short TX	Cross-service, long-running
Blocking	Yes (coordinator crash)	No

Interview takeaway: "2PC gives strong consistency but blocks on coordinator failure. Saga gives eventual consistency with better availability. For microservices, Saga is preferred. The cost: you must design compensating transactions and accept temporary inconsistency."

5.12 Replication Strategies

Strategy	How	Pros	Cons
Single-Leader	All writes to primary, async/sync replication to replicas	Simple, strong consistency possible	Leader = SPOF, replica lag
Multi-Leader	Multiple leaders, conflict resolution needed	Geo-distribution, resilient	Write conflicts (last-write-wins, CRDTs, merge)
Leaderless (Dynamo)	Client writes to N nodes directly, quorum	High availability, no leader SPOF	Conflict resolution (vector clocks), stale reads

Strategy	How	Pros	Cons
Chain Replication	Head accepts writes, passes through chain, tail acknowledges to client	Strong consistency, good throughput	Write latency = chain length, tail is read SPOF

Replication lag issues:

- **Read-after-write:** User writes, then reads from replica before write propagates
- **Monotonic reads:** User reads from replica A (sees new data), then from replica B (sees old data)
- **Causality violations:** User sees reply before the post it replies to

Solutions: sticky sessions, read from leader after write, version-based consistency.

5.13 Gossip Protocol

Concept: Nodes periodically select random neighbors and exchange state information. Epidemiological model — information spreads like a virus.

```
Step 0: Node A has new info
        [A*] [B] [C] [D] [E] [F]

Step 1: A gossips to C and E
        [A*] [B] [C*] [D] [E*] [F]

Step 2: C gossips to B; E gossips to D and F
        [A*] [B*] [C*] [D*] [E*] [F*]

All nodes informed in O(log N) rounds
```

Properties:

- **Epidemic spread:** O(log N) rounds to reach all N nodes
- **Resilient:** Works even if many nodes fail
- **Eventually consistent:** No guarantees on timing
- **Scalable:** Each node makes constant number of contacts per round

Use cases:

- Membership/failure detection (Cassandra, Riak)
- State dissemination (config updates)
- Anti-entropy (reconcile differences between replicas)

Interview takeaway: "Gossip provides decentralized, fault-tolerant information propagation. Cassandra uses gossip for cluster membership. It's eventually consistent — nodes might have temporarily stale views."

5.14 Heartbeats and Failure Detection

Challenge: In asynchronous networks, you cannot distinguish a slow node from a dead one (FLP impossibility).

Heartbeat: Nodes send periodic "I'm alive" messages. Timeout → assume dead.

Problems:

- Short timeout: false positives (network hiccup = assume dead)
- Long timeout: slow detection of actual failures

Phi Accrual Failure Detector (Cassandra): Instead of binary live/dead, outputs a suspicion level φ (phi). φ increases the longer since last heartbeat. Application sets threshold: if $\varphi > 8$, assume dead. Adapts to changing network conditions.

Split-Brain: When a network partition isolates part of a cluster, both sides may believe they're the leader. Both sides serve writes → data diverges → nightmare.

```
Before partition:               After partition:
  [Leader] ——— [F1]               [Leader] —X— [F1]
               [F2]                            [F2]

Both sides think they're primary. Writes diverge.
```

Prevention:

- **STONITH (Shoot The Other Node In The Head):** Active fencing — force the other node offline
- **Quorum-based leadership:** Leader requires majority quorum. Minority partition cannot elect leader.
- **Epoch/term numbers:** New leader always has higher term. Old leader rejects its own writes when it sees higher terms.

5.15 Service Discovery

Problem: In dynamic environments (containers, auto-scaling), service IPs change constantly. Clients can't use static IPs.

Two patterns:

Client-Side Discovery:

```
Client → Query Registry (etcd/Consul/ZooKeeper) → Get list of instances
Client → Load balances itself → Picks instance → Direct call
```

- Netflix Eureka + Ribbon (client-side)
- Pro: Client controls load-balancing algorithm
- Con: Discovery logic in every client (language-specific)

Server-Side Discovery:

```
Client → Load Balancer (ELB, Nginx, Envoy)
Load Balancer → Query Registry → Pick instance → Forward
```

- AWS ELB + Route 53
- Pro: Simple client, language-agnostic
- Con: Load balancer is another hop + potential SPOF

Service Registry Types:

- **ZooKeeper:** CP, strong consistency, general-purpose coordination
- **Consul:** CP, built for service discovery + health checks + KV
- **etcd:** CP, key-value, Kubernetes backing store
- **Eureka:** AP, built for availability, used by Netflix

Health checking approaches:

- **Active:** Registry pings service (HTTP /health)
- **Passive:** Service sends heartbeats to registry (TTL-based)
- **Ephemeral leases:** If service dies, lease expires and entry removed

5.16 Distributed Caching

Cache patterns:

Pattern	How	Use Case
Cache-Aside (Lazy Loading)	App checks cache, on miss loads from DB and populates cache	General purpose; stale data possible
Write-Through	Write to cache AND DB synchronously	Strong consistency; higher write latency
Write-Behind (Write-Back)	Write to cache, async write to DB	High write throughput; risk of data loss
Read-Through	Cache handles DB reads automatically	Transparent to app; must configure
Refresh-Ahead	Pre-populate cache before expiry	Low miss rate; wastes memory if unused

Cache consistency problems:

- **Cache stampede / thundering herd:** Cache expires, all requests hit DB simultaneously. Solution: probabilistic early expiration, mutex/lock on miss
- **Cache invalidation:** "There are only two hard things in CS: cache invalidation and naming things" — Phil Karlton. Solutions: TTL, event-based invalidation (CDC), versioning
- **Hotspot keys:** One popular key overloads one cache node. Solutions: local in-process cache, key replication across nodes, consistent hashing with virtual nodes

Distributed cache implementations:

- **Memcached:** Simple KV, no persistence, client-side sharding, very fast
- **Redis:** Rich data structures (lists, sets, sorted sets), optional persistence, Lua scripting, Redis Cluster for sharding

Cache eviction policies:

Policy	Evicts	Good for
LRU	Least recently used	General; protects frequently-used items
LFU	Least frequently used	Protects popular items better than LRU
FIFO	Oldest inserted	Simple; ignores access patterns
Random	Random item	Surprisingly OK; simple to implement
TTL	Expired items	Time-sensitive data

5.17 Kafka

Kafka — distributed event streaming platform. Originally LinkedIn, now Apache.

Core concepts:

- **Topic:** Named stream of records. Like a database table but append-only and distributed.
- **Partition:** Topics are split into partitions. Each partition is an ordered, immutable log. Enables parallelism.
- **Broker:** Kafka server. A cluster has multiple brokers.
- **Producer:** Writes records to topics (chooses partition via key hash or round-robin).
- **Consumer:** Reads records from partitions by maintaining an **offset**.
- **Consumer Group:** Group of consumers sharing a topic. Each partition consumed by exactly one consumer in the group.

- **Offset:** Sequence number of a record within a partition. Consumer controls its own offset.
- **Replication Factor:** How many copies of each partition. Leader partition handles reads/writes; followers replicate.
- **ISR (In-Sync Replicas):** Replicas caught up with the leader. Leader waits for ISR to ACK before committing (configurable).

Delivery semantics:

Semantic	Config	Meaning
At-most-once	acks=0, no retry	May lose messages; never duplicates
At-least-once	acks=all, retry enabled	No loss; may duplicate
Exactly-once	Idempotent producer + transactions	No loss, no duplicates (within Kafka)

Exactly-once is a myth at system level — even with Kafka's exactly-once, if your consumer's side effects are non-idempotent, you can still cause duplicates. The key: **make consumers idempotent**.

Why Kafka is fast:

1. **Sequential disk writes** — OS page cache, no random seeks
2. **Zero-copy** (sendfile syscall) — data goes from disk to socket without copying to user space
3. **Batching** — producer and broker batch records, amortize overhead
4. **Log compaction** — keeps only latest value per key; enables changelog topics

Kafka ordering guarantee: Within a partition, records are strictly ordered. Across partitions, no global order guarantee. This is why key selection matters: all messages for a user should go to the same partition (keyed by userId).

Consumer group rebalancing: When consumers join/leave, partitions are reassigned. All consumers stop processing during rebalance (stop-the-world). Kafka 2.4+ introduced cooperative rebalancing to minimize disruption.

5.18 Idempotency and Exactly-Once vs At-Least-Once

Idempotent operation: Applying it multiple times has the same effect as applying once.

```
Idempotent:     SET x = 5           → safe to retry
Non-idempotent: INCREMENT x         → retry causes overcounting

Making it idempotent: SET x = 5 WHERE version = 3
```

Idempotency key: Client sends a unique ID with each request. Server deduplicates.

```
POST /payments
{
  "idempotency_key": "a1b2c3-uuid",
  "amount": 100,
  "account": "123"
}
Server stores (idempotency_key → result) and returns cached result on retry.
```

At-least-once vs Exactly-once:

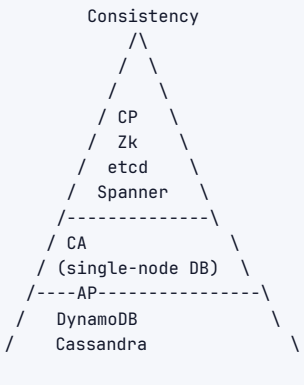
Aspect	At-Least-Once	Exactly-Once
Risk	Duplicate processing	None in theory
Complexity	Low	High

Aspect	At-Least-Once	Exactly-Once
Performance	Better	Worse (more coordination)
Best practice	Make consumers idempotent	Only when truly needed
Real-world usage	Most systems	Payment systems, financial

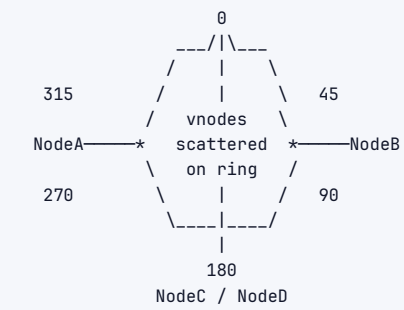
Interview takeaway: "Exactly-once is extremely hard to achieve end-to-end. At-least-once + idempotent consumers is the practical solution in >95% of systems. Design your consumer operations to be naturally idempotent or add idempotency keys."

6 Architecture / Diagrams

6.1 CAP Triangle



6.2 Consistent Hashing Ring




```
Example positions:
NodeA: 15, 120, 240
NodeB: 60, 190, 310
NodeC: 90, 210, 350
NodeD: 30, 150, 270

Key at hash 100 → next clockwise ≥ 100 → 120 (NodeA)
Key at hash 200 → next clockwise ≥ 200 → 210 (NodeC)
```

6.3 Raft: Leader Election and Log Replication

```
INITIAL STATE:                    AFTER ELECTION:
[F: term=1]                       [L: term=2]  ← elected leader
[F: term=1]  → timeout →          [F: term=2]
[F: term=1]                       [F: term=2]

LEADER ELECTION MESSAGES:
Candidate → RequestVote(term=2) → All
Each Follower → VoteGranted       → Candidate (if log OK, first vote this term)
Candidate receives 2 of 3 votes → becomes Leader

LOG REPLICATION:
Client → SET x=5 → Leader

Leader Log:  | idx=1,t=1,x=1 | idx=2,t=2,x=5 | ← new, uncommitted
                                    |
                              AppendEntries RPC
                             /                 \
                            ↓                   ↓
Follower1 Log: |idx=1,t=1| |idx=2,t=2,x=5|   Follower2 Log: |idx=1,t=1| |idx=2,t=2,x=5|
                             ↑ ACK                            ↑ ACK

Leader receives 2 ACKs (majority of 3) → commits idx=2
→ Responds to client: OK
→ Next heartbeat tells followers: commit up to idx=2
Followers apply idx=2 to state machine

SAFETY: Follower only votes for candidate whose log is at least as up-to-date
        This prevents any node from winning without all committed entries
```

6.4 Quorum Reads and Writes (N=5, R=3, W=3)

```
WRITE (W=3):
Client → Write(x=5) → N1, N2, N3, N4, N5
                       ↑    ↑    ↑
                      ACK  ACK  ACK  ← 3 ACKs (majority) = committed
                                N4, N5 may respond later

READ (R=3):
Client ← Read(x) ← N1, N2, N3
         |          x=5  x=5  x=3 ← N3 is stale
         |
         Conflict resolution: pick latest version (vector clock / timestamp)
         Return x=5

OVERLAP GUARANTEE:
Write set {N1,N2,N3}, Read set {N1,N2,N3}: Overlap = {N1,N2,N3}
At least one node in read set always has the latest write. ✓
```

6.5 Kafka Topic/Partition/Consumer-Group

```
Topic: "user-events"  (3 partitions, replication-factor=2)

Partition 0: [msg0][msg1][msg4][msg7]...  Leader: Broker1, Replica: Broker2
Partition 1: [msg2][msg5][msg8]...        Leader: Broker2, Replica: Broker3
Partition 2: [msg3][msg6][msg9]...        Leader: Broker3, Replica: Broker1

Producer A (key=userId % 3):
  userId=10 → partition 1
  userId=20 → partition 2
  userId=30 → partition 0

Consumer Group "analytics":
  Consumer X → reads Partition 0 (offset: 7)
  Consumer Y → reads Partition 1 (offset: 8)
  Consumer Z → reads Partition 2 (offset: 9)
```

```
Consumer Group "notifications":
  Consumer P → reads Partition 0 + Partition 1
  Consumer Q → reads Partition 2

Rule: Each partition is consumed by exactly ONE consumer per consumer group
      Multiple consumer groups each see ALL messages independently
```

7 Real-World Examples

System	CAP	Consistency	Sharding	Notes
DynamoDB	AP	Eventual (default), tunable	Consistent hashing	Dynamo paper: vector clocks, sloppy quorum, hinted handoff
Cassandra	AP	Tunable (ONE to ALL)	Consistent hashing + vnodes	Gossip for membership, LWW for conflict resolution
Spanner	CP	External consistency (linearizable + serializable)	Sharded Paxos	TrueTime API bounds clock uncertainty
ZooKeeper	CP	Sequential consistency	No (single-leader)	ZAB protocol (Paxos-like), used for coordination/leader election
etcd	CP	Linearizable	No (single-leader, but clustered)	Raft consensus, Kubernetes backing store
HBase	CP	Strong (via HDFS + ZooKeeper)	Region-based	Built on Hadoop, ZooKeeper for master election
MongoDB	CP (default)	Causal (tunable)	Range-based sharding	Primary-secondary replication, elections on failure
Redis Cluster	AP	Eventual (async replication)	Hash slots (16384)	Sentinel for HA, Cluster for sharding
Kafka	CP (for partition leadership)	Per-partition order	Log partitioning	ISR-based replication, ZooKeeper → KRaft migration
Memcached	AP	None (no replication)	Client-side consistent hash	Pure cache, no HA by itself

8 Real-Life Analogies

One kingdom of villages, connected only by riders — every concept is a village, a decree, or a messenger.

Concept	Analogy
CAP Theorem	When the road between regions is washed out (partition), the council either halts all decisions until it reopens (consistency) or lets each region rule on its own possibly-stale information (availability) — it cannot do both at once.

Concept	Analogy
Eventual Consistency	A decree carried by riders to every village: for a while different villages know different things, but the news eventually reaches all, and the kingdom settles on the same truth.
Consistent Hashing	Granaries arranged around the kingdom's ring road, each village responsible for the stretch of road after it. When one village abandons its granary, only that stretch is reassigned to the next village — the rest of the kingdom's ledgers stay untouched.
Raft Leader Election	If the chief goes silent, a candidate rides out campaigning village to village; once a majority of village elders pledge support, that candidate is crowned chief for a new reign (the "term number"). Any rival who sees a higher reign number stands down immediately.
Quorum	A decree becomes law only if a majority of village elders vote yes — even with some elders absent on harvest duty, the decree can still pass as long as more than half of all elders have sealed it.
Vector Clocks	Each rider's message notes "I'd heard up to your 3rd decree and the western village's 2nd." You can tell which decree answered which, and when two villages issued orders without hearing from each other, their clocks reveal the messages as concurrent — no one can claim one came first.
Gossip Protocol	A rider arrives at one village with news; that village's own riders carry it to two neighbours next dawn; each of those villages dispatches riders in turn. In a few rounds every corner of the kingdom has heard — even if some riders never arrive, enough paths exist that the news still spreads.
Saga Pattern	A multi-village grain trade: Village A ships wheat, Village B grinds it to flour, Village C bakes bread for market. If the miller's fire burns down mid-agreement, the miller sends word back to cancel the wheat shipment and refund the delivery riders — each step carries a written undo-scroll so the trade can be unwound cleanly.
2PC	The chief sends a rider to every village asking "are you ready to commit to the harvest levy?" (Phase 1: Prepare). Only when all villages reply "ready" does a second rider ride out saying "commit!" If any village replies "not ready" — or the chief's rider is lost mid-journey — every village holds its grain locked in the storehouse, waiting, unable to trade until the chief's next decree arrives.
Split Brain	A flood cuts the kingdom's roads in two. Both halves believe their regional elder is now chief, both issue decrees, both stamp their own ledgers as authoritative. When the roads reopen, the scribes find two contradictory sets of laws and must painfully reconcile whose decrees count — or one chief must abdicate.
Heartbeat	Each village elder sends a rider to the chief every dusk carrying only a blank scroll that means "still here." If the chief receives no blank scroll from a village for three dawns running, the council assumes that village has gone dark and dispatches a relief rider to investigate.
Idempote	A rider delivers the same levy decree twice because the road was muddy and they feared the first had been lost. A wise village elder stamps the decree's unique seal number in the ledger: if the seal has already been recorded, the levy is not collected a second time — the kingdom's accounts stay sound no matter how many copies arrive.

9 Memory Tricks / Mnemonics

9.1 CAP: "Pick 2, but P is not optional"

```
CAP = C A P
      ↑ ↑ ↑
      | | └── Partition Tolerance (MANDATORY in real systems)
      | └──── Availability (return a response)
      └────── Consistency (latest data)
```

```
Real choice: CP (return error) or AP (return stale data)
```

9.2 PACELC: "P picks A or C, ELse picks L or C"

```
P → A/C (partition scenario)
ELC → L/C (normal scenario)
"During Partition = AC, Else = LC"
```

9.3 Raft: "FELT" for leader election

```
F - Follower times out
E - Election begins (become Candidate)
L - Log check (only vote if candidate log ≥ mine)
T - Term number (higher term always wins)
```

9.4 Quorum: R + W > N

```
N = 5, want quorum: R=3, W=3 → 3+3=6 > 5 ✓
"Read + Write must exceed total replicas"
Fast reads: R=1, W=5 → write to all
Fast writes: R=5, W=1 → read from all
```

9.5 Consistency Levels (strongest to weakest):

```
"Lions Shouldn't Carry Raw Meat Ever"
L - Linearizable
S - Sequential
C - Causal
R - Read-your-writes
M - Monotonic reads
E - Eventual
```

9.6 8 Fallacies: "NLB-STT-Z"

```
N - Network is reliable
L - Latency is zero
B - Bandwidth is infinite
S - Secure network
T - Topology doesn't change
T - There is one admin
Z - Zero transport cost
H - Homogeneous network
```

(Or remember: "No Legitimate Business Survives Thinking They're Zeus-like Homogeneous")

9.7 2PC vs Saga

```
2PC = "2 Phases, Can block" → ACID, tight coupling, blocking
SAGA = "Steps And Compensating Actions" → eventual, loose, non-blocking
```

9.8 Raft Safety in one line:

```
"A leader can only win if its log is as complete as the majority"
→ Ensures all committed entries survive leader changes
```

10 Common Interview Questions

10.1 Q1: "What is the CAP theorem? Give examples."

Model answer: ”CAP states that in the presence of a network partition, a distributed system must choose between consistency and availability. Partition tolerance isn't optional — networks partition, so you always pick CP or AP.

CP example: ZooKeeper returns errors during partitions rather than serving stale data — used where coordination correctness matters (leader election, distributed locks).

AP example: Cassandra and DynamoDB return potentially stale data during partitions — used where availability matters more than perfect consistency (shopping carts, social feeds).

Important nuance: CAP only describes behavior during a partition. PACELC extends this by noting that even during normal operation, you trade latency against consistency.”

Follow-ups:

- ”What's the difference between linearizability and sequential consistency?” → Linearizability: real-time ordering constraint. Sequential: some valid ordering consistent per-process.
- ”Is Cassandra really AP?” → Tunable: at consistency level ONE it's AP, at ALL it's effectively CP
- ”What's PACELC?” → See PACELC section above

10.2 Q2: "Explain how Raft works."

Model answer: ”Raft is a consensus algorithm that replicates a state machine log across a cluster. It has three key components:

Leader election: Nodes start as followers. On timeout (randomized 150-300ms), a follower becomes a candidate, increments its term, and requests votes. A node wins if it gets a majority, and only if its log is at least as up-to-date as each voter's log. The winning node becomes leader for that term.

Log replication: All writes go to the leader. The leader appends the entry locally, then sends AppendEntries RPCs to all followers. Once a majority acknowledges, the leader commits the entry, responds to the client, and notifies followers to commit in the next heartbeat.

Safety: The election rule ensures a new leader always has all committed entries — committed means majority wrote it, and a majority must vote, so their intersection always has the committed entry. This is the Log Matching property.

Raft guarantees: leader completeness (elected leader has all committed entries), log matching (identical index+term means identical logs up to that point), and state machine safety (all nodes apply the same log in the same order).”

Follow-ups:

- ”What happens when the leader crashes?” → New election. Any committed entry survives because the winner has all committed entries.
- ”What about uncommitted entries in leader's log?” → They may or may not survive. Raft only guarantees committed entries survive.
- ”How does Raft handle log conflicts?” → AppendEntries includes prevLogIndex and prevLogTerm. Follower rejects if mismatch. Leader walks back until they agree, then overwrites.

10.3 Q3: "Design a distributed cache."

Model answer: "I'd use a consistent hashing ring with virtual nodes for data distribution across N cache nodes. For high availability, replicate each key to the next K nodes clockwise.

Client routing: Use client-side consistent hashing (memcached ketama) or a proxy layer (like mcrouter or Envoy with hashing LB).

Cache operations:

- Cache-aside: application checks cache, misses hit DB, then populate cache
- TTL-based expiration with lazy eviction + background cleanup
- LRU or LFU eviction policy

Failure handling:

- Node failure: consistent hashing routes to next node; replication ensures data survives
- Cache stampede on miss: use probabilistic early expiration or a distributed mutex (Redis SET NX with TTL)

Consistency:

- Write-through for critical data (payment status)
- Write-behind for high-throughput writes (analytics events)
- Event-based invalidation via CDC for DB-cache sync

At scale (e.g., Meta's Memcached), layers: L1 in-process cache → L2 regional Memcached cluster → DB. Replication groups for high read throughput."

10.4 Q4: "Explain consistent hashing and why it's used."

Model answer: "Consistent hashing places nodes and keys on a virtual ring from 0 to 2^32. A key is assigned to the nearest node clockwise. When a node joins or leaves, only O(K/N) keys — the ones between the departing node and its predecessor — are remapped, vs O(K) for naive modular hashing.

Without it, adding one node to a 10-node cluster would remap ~90% of keys, causing a cache avalanche as every remapped key misses and hits the database.

Virtual nodes improve load distribution: each physical node has multiple ring positions, so adding or removing a node draws/redistributes keys from many neighbors rather than just two."

10.5 Q5: "What is idempotency and why does it matter?"

Model answer: "An idempotent operation produces the same result whether applied once or multiple times. It matters because in distributed systems, network failures cause retries. Without idempotency, retries cause double-charges, duplicate orders, or corrupted state.

Implementation: idempotency keys. The client sends a UUID with each request. The server stores (key → response) and returns the cached result on retry. The key must be scoped appropriately — per-user, per-resource, with TTL.

In Kafka: idempotent producers use a PID + sequence number. Broker deduplicates retried batches. Transactional producers extend this across partitions.

The practical rule: at-least-once delivery + idempotent consumers = effectively-once processing, without the complexity of true exactly-once."

10.6 Q6: "What is split-brain and how do you prevent it?"

Model answer: "Split-brain occurs when a network partition causes two sides of a cluster to each believe they're the active leader. Both accept writes, creating divergent state that's extremely hard to reconcile.

Prevention strategies:

1. **Quorum-based leadership:** Leader requires majority (N/2+1) acknowledgment. Minority partition can't elect a leader, so only one side stays active.
2. **Epoch/term fencing:** Leader writes include its term. If old leader comes back after partition, followers reject its writes because they've seen a higher term.
3. **STONITH (Shoot The Other Node In The Head):** Actively fence the other node — cut its power, revoke its network access — before proceeding. Used in HA database clusters.
4. **Witness nodes:** A tie-breaking node that grants quorum to one side.

In Raft: the minority-side leader steps down when it can no longer replicate to a majority and stops receiving heartbeats from the new leader."

11 Senior-Level Discussion Points

11.1 Why Paxos Is Hard to Implement

1. **Multi-Paxos vs Single-Decree:** The original paper only describes agreeing on ONE value. Real systems need an ordered log (Multi-Paxos). The paper doesn't specify this, leaving huge implementation gaps.
2. **Liveness holes:** Two proposers with consecutive proposal numbers can livelock forever (each preempts the other). Need a leader, but Paxos doesn't specify leader election.
3. **Log gaps:** If an instance has no accepted value, you need to fill it (no-op). Non-trivial.
4. **Leader leases:** Reading committed state without contacting a majority on every read requires leader leases — another protocol layered on top.
5. **Reconfiguration:** Changing cluster membership is extremely tricky.

Google's Chubby paper (2006): "there are significant gaps between the description of the Paxos algorithm and the needs of a real-world system... the final system will be based on an unproven protocol."

11.2 The Exactly-Once Myth

True exactly-once delivery at the network layer is **impossible** in the face of node failures (you can't know if a message was delivered or not without asking, which risks duplicates). What systems achieve:

- **Kafka exactly-once semantics:** Exactly-once within Kafka's producer-broker-consumer pipeline (idempotent producers + transactional APIs). But if the consumer writes to a non-transactional external system (DB, HTTP API), you're back to at-least-once.
- **Practical exactly-once:** Kafka + idempotent consumer with a transactional DB. Write the output AND commit the Kafka offset in one DB transaction. If processing fails, rollback both. But requires the consumer's output to be in the same transactional store as the offset.

11.3 Trade-offs: Consistency vs. Latency

- Every synchronous replication round trip adds ~1ms per hop, per replica. Cross-region = 50-150ms per round trip.
- Spanner achieves external consistency but sacrifices latency: ~5-10ms per write vs DynamoDB's ~1ms
- For social media (likes, view counts), eventual consistency is fine — users won't notice 100ms convergence delay
- For financial systems (balances, inventory), strong consistency is required — business logic demands it

11.4 Failure Scenarios to Discuss

1. **Leader with stale followers:** Network partition isolates leader. Minority leader keeps accepting writes. When partition heals, those writes are lost (overwritten by new majority's leader).
2. **Thundering herd on cache miss:** Cache node fails. All clients simultaneously miss and stampede the DB. Mitigation: circuit breaker, request coalescing, probabilistic TTL extension.
3. **Slow node in quorum:** If you need 3 of 5 nodes to ACK, one slow node adds its latency to all writes. Dynamo uses ”fastest R of N” and ”background write” to avoid this.
4. **Zombie leader:** Old leader (post-partition) still thinks it's leader. Without fencing tokens, it may service reads (returning stale data) or attempt writes (rejected by followers who see higher term). Solution: epoch-based fencing, STONITH.
5. **Cascading failure:** Node A fails → Node B takes on its load → B becomes overloaded → B fails → cascade. Mitigation: load shedding, bulkheads, backpressure.

11.5 Gossip vs Consensus for Membership

- **Gossip (Cassandra):** Eventually consistent membership. Nodes may briefly disagree on who's in the cluster. Cheap, scales to thousands. Fine for eventual-consistency systems.
- **Consensus (ZooKeeper, etcd):** Strongly consistent membership. All nodes agree. Slower (requires majority ACK). Required for systems that need exactly-once leader election or distributed locks.

12 Typical Mistakes Candidates Make

Mistake	Correction
”CA is a valid CAP choice”	CA is only meaningful on a single machine. Distributed systems must tolerate partitions. Real choice is CP vs AP.
”Eventual consistency = broken consistency”	Eventual consistency is appropriate and correct for many use cases (DNS, social feeds, analytics). It's a deliberate trade-off.
Confusing **linearizability** with **serializability**	Linearizability: single-operation real-time ordering. Serializability: transaction-level equivalence to serial execution. Spanner has BOTH (strict serializability).
Saying ”Kafka guarantees exactly-once delivery”	Kafka guarantees exactly-once within its own pipeline. End-to-end exactly-once requires idempotent consumers + transactional processing.
”Just use 2PC for distributed transactions”	2PC blocks on coordinator failure. For microservices, Saga with compensating transactions is more appropriate.
”Raft is just like Paxos”	Raft is simpler and more understandable. Key differences: strong leader, sequential log (no holes), joint consensus for config changes.
Not mentioning failure scenarios	Always discuss: what happens when a node dies, when network partitions, when the leader crashes. Interviewers want failure mode awareness.
Treating CAP as binary switches	Tunable consistency (Cassandra) shows it's a dial, not a switch. Different operations in the same system can have different consistency levels.
”More replicas = more availability”	More replicas also means more nodes to fail during synchronous writes. Must balance replication factor with write latency and failure probability.

Mistake	Correction
Forgetting clock skew	Always mention that distributed timestamps require care. Lamport clocks, vector clocks, or TrueTime. Never assume clocks are synchronized.

13 How This Connects To Other Topics

13.1 System Design

- Every large-scale system design requires: partitioning strategy (consistent hashing or range), replication strategy (leader-follower or leaderless), consistency model choice, and service discovery
- ”Design a URL shortener” → need distributed ID generation (Snowflake), caching (Redis consistent hash), and AP storage (Cassandra)
- ”Design WhatsApp” → Kafka for message queuing, consistent hashing for user routing, read-your-writes consistency for messages

13.2 Databases

- SQL ACID transactions = linearizability + serializability = CP on single node
- Distributed SQL (Spanner, CockroachDB) = Paxos/Raft + 2PC for cross-shard transactions
- NoSQL tradeoffs: MongoDB (CP, tunable), Cassandra (AP, tunable), DynamoDB (AP, tunable)
- Replication lag (MySQL async replication) = eventually consistent reads from replicas
- MVCC + snapshot isolation = consistency model at DB level

13.3 Computer Networks

- TCP reliability helps but doesn't solve distributed consensus (still need application-level ACKs)
- Network partition = the P in CAP. VPCs, AZs, regions add partition probability
- HTTP idempotency: GET/PUT/DELETE are idempotent by design, POST is not
- gRPC / Protobuf: reduce serialization cost (fallacy #7)
- TLS/mTLS: solve fallacy #4 (network security)

13.4 Concurrency

- Distributed locks are the distributed equivalent of mutexes
- Compare-and-Swap (CAS) at DB level enables optimistic locking = distributed equivalent of lock-free algorithms
- Thread safety in single process :: consensus in distributed system
- Race conditions in distributed systems → addressed by ordering guarantees, vector clocks, transactions
- Deadlock → distributed deadlock detection is harder (requires global knowledge)

14 FAANG Interview Tips

For system design rounds:

1. **Always ask about consistency requirements** early. ”Does this need strong consistency, or is eventual OK?” This shapes all subsequent decisions.
2. **Name the patterns explicitly:** ”I'd use consistent hashing for this because...” shows vocab-

ulary.
3. **Discuss failure modes proactively:** "If the leader crashes during this write, here's what happens..." — separates senior candidates.
4. **PACELC lens:** After choosing CP/AP for partition scenario, immediately discuss "In normal operation, we're trading latency for consistency by..."
5. **Quantify:** "With R=2, W=2, N=3, we get majority quorum on both reads and writes. Write latency is ~2x a single node write because we wait for both ACKs."

For depth/DS&A rounds on distributed topics:

1. Know Raft cold — leader election, log replication, safety. Draw the state machine.
2. Know the Dynamo paper highlights — consistent hashing, vector clocks, sloppy quorum, hinted handoff
3. Know Kafka deeply — partitions, consumer groups, offset management, ISR, delivery semantics
4. Be prepared to critique trade-offs rather than just describe systems

Communication patterns:

- "The classic tension here is X vs Y. If we prioritize X, we'd choose Z. If we prioritize Y, we'd choose W. Given the requirements you mentioned, I'd lean toward..."
- "This is the split-brain problem. The standard solution is quorum-based leadership, specifically..."
- "This reminds me of how DynamoDB handles this — they use sloppy quorum with hinted handoff to maintain availability during partitions."

Red flags to avoid:

- Saying "just use a database" without specifying consistency/replication model
- Not knowing the difference between a Kafka topic, partition, and consumer group
- Unable to explain why Paxos is hard to implement
- Treating distributed systems as deterministic/reliable

15 Revision Cheat Sheet

15.1 10-Minute Summary

1. **CAP:** During partition, choose Consistency (error) or Availability (stale). P is mandatory.
2. **PACELC:** Extends CAP to normal ops: choose Latency or Consistency on every request.
3. **Consistency models:** Linearizable > Sequential > Causal > Read-your-writes > Eventual
4. **Consistent hashing:** O(K/N) remapping when nodes change. Virtual nodes for even distribution.
5. **Quorum:** R+W>N guarantees overlap. Sloppy quorum trades consistency for availability.
6. **Raft:** Leader election (majority votes, log must be up-to-date) + Log replication (leader commits on majority ACK). Term = epoch.
7. **Paxos:** Two phases (Prepare/Accept). Proven correct, painful to implement. Raft is practical alternative.
8. **Vector clocks:** Detect causality. V_a < V_b → a happened-before b. Otherwise concurrent.
9. **2PC:** ACID across nodes but blocks on coordinator failure. Use Saga for microservices.
10. **Kafka:** Topics → Partitions (ordered log). Consumer groups (each partition → 1 consumer per group). ISR for replication.
11. **Idempotency:** At-least-once + idempotent = effectively exactly-once. Use idempotency keys.

12. **Split-brain:** Two leaders diverge. Prevent with quorum, epoch fencing, STONITH.
13. **Gossip:** O(log N) propagation, resilient, eventually consistent. Used for membership.
14. **Service discovery:** Client-side (Eureka) vs server-side (ELB). Registry must have health checks.

15.2 Key Points

- **The 8 fallacies:** Network unreliable, latency not zero, bandwidth not infinite, not secure, topology changes, multiple admins, transport has cost, heterogeneous
- **CP systems:** ZooKeeper, etcd, Spanner, HBase, MongoDB
- **AP systems:** DynamoDB, Cassandra, CouchDB, DNS
- **Raft safety guarantee:** Leader always has all committed entries (election log check)
- **Kafka exactly-once:** Only within Kafka pipeline; consumers must be idempotent for E2E
- **Consistent hashing:** Used in DynamoDB, Cassandra, Memcached, Redis Cluster, CDNs
- **2PC vs Saga:** 2PC = strong, blocking; Saga = eventual, non-blocking, compensating transactions

15.3 Comparison Tables Cheat Sheet

CAP:

```
CP: error on partition | AP: stale data on partition
ZooKeeper, etcd        | Cassandra, DynamoDB, DNS
```

Quorum:

```python
R+W>N = strong consistency guarantee
R=1, W=N: fast reads | R=N, W=1: fast writes | R=W=(N+1)/2: balanced
```

Raft vs Paxos:

```
Raft: explicit leader, sequential log, understandable, etcd/CockroachDB
Paxos: implicit leader, log holes, complex, Chubby/Spanner
```

2PC vs Saga:

```
2PC: ACID, blocking, tight coupling, same DB
Saga: eventual, non-blocking, loose coupling, microservices
```

At-least-once vs Exactly-once:

```
At-least-once: simple, may duplicate → make consumers idempotent
Exactly-once: complex, within Kafka only → use idempotency keys end-to-end
```

Delivery semantics:

```python
At-most-once: acks=0, may lose, no duplicates
At-least-once: acks=all + retry, no loss, may duplicate
Exactly-once: idempotent producer + transactions (Kafka internal)
```

15.4 Most Important Concepts (Priority Order)

1. CAP + PACELC (asked in every system design)
2. Raft consensus (frequently deep-dived)
3. Consistent hashing (appears in cache/DB designs)
4. Quorum R+W>N (core to DynamoDB/Cassandra discussion)
5. Kafka architecture (streaming systems everywhere)
6. Consistency models (senior-level differentiation)
7. Saga vs 2PC (microservices design)
8. Idempotency (practical distributed programming)
9. Vector clocks (conflict detection)
10. Split-brain + failure detection (operational maturity signal)

Study tip: For each major concept, be able to (1) define it in one sentence, (2) give a real system example, (3) explain the trade-off, and (4) describe a failure scenario. This structure works for any distributed systems question.